

Pg_repack using gitlab-pgrepack

Here the link to the video of the runbook simulation.

gitlab-pgrepack is a helper tool for running pg_repack in GitLab environments. It is open source and works on top of the pg_repack PostgreSQL program.

What is bloat?

Bloat can be seen as the “remains” of UPDATE and DELETE activity. It is important to distinguish the concepts of “dead tuples” and “bloat”. When a row is updated or deleted, PostgreSQL creates new physical versions of rows – called “tuples”. After transaction finishes, old tuples are marked as “dead”. Accumulation of dead tuples (caused by a massive operation or temporary autovacuum’s inability to clean up dead tuples promptly) leads to the situation, when autovacuum cleans many dead tuples at once, leaving significant “gaps” in physical layout (empty space in pages) – these gaps is what we call “bloat”.

Both tables and indexes can be bloated. Two negative effects of the growing bloat are:

1. excessive disk space usage,
2. performance degradation (usually, high levels of index bloat are more dangerous for performance).

Why pg_repack?

VACUUM FULL is a standard command that can be used to get rid of bloat. But, as opposed to regular VACUUM (executed either by autovacuum or manually), VACUUM FULL uses a long-lasting exclusive lock on the table (basically, recreating the table and its indexes), blocking all the queries to the table and impacting performance. Pg_repack uses an approach that requires short-lived locks, being obtained gracefully (with multiple attempts with `statement_timeout` gradually growing from 100ms to 1000ms), suitable for databases with high and concurrent activity.

How to see estimated bloat for tables and indexes

- In monitoring: PostgreSQL Bloat Dashboard
- postgres-checkup reports – see reports F004 Autovacuum: Heap Bloat (Estimated) and F005 Autovacuum: Btree Index Bloat (Estimated). See Checking PostgreSQL health with postgres-checkup for more information.

What to repack

Bloat cannot be seen in exact numbers without expensive reading of database files, but we can have light checks providing estimates (which may be off up to

dozens of percent in some cases, however). Usually, we consider bloat level of 20-30% as “normal”, 50-60% as “significant”, 80-90% as “high”, and >90% as “extremely high”.

Generally, we should start considering repacking indexes when the estimated bloat exceeds 50%, and definitely need to repack when it is greater than 60%. For tables, it depends: if we know that soon more data will be inserted to the table, we can skip repacking tables with a high level of bloat (two reasons: repacking of tables is lock-heavy, and also high table bloat usually doesn’t affect performance directly), and we definitely should consider repacking tables with estimated bloat 90% and higher.

Of course, the repacking of tiny objects (such as tables or indexes which size is a few MiB) may not make sense and can be safely skipped.

Prerequisites

- Request an auth token for Grafana annotations if needed and modify `auth_key` accordingly.

Setting up the tool

Install dependencies, postgresql-17-repack, libpq and Ruby 3.1.2 (using RVM) in the console node:

```
sudo apt install gnupg2
sudo apt install postgresql-17-repack
sudo apt install libpq-dev
```

```
gpg2 --recv-keys 409B6B1796C275462A1703113804BB82D39DC0E3 7D2BAF1CF37B13E2069D6956105BD0E73
\curl -sSL https://get.rvm.io | bash -s stable
source ~/.rvm/scripts/rvm
rvm install 3.1.2
```

Install gitlab-pgrepack tool in your \$HOME:

```
cd $HOME
```

```
git clone https://gitlab.com/gitlab-com/gl-infra/gitlab-pgrepack.git
cd gitlab-pgrepack
rvm use 3.1.2
gem install bundler
bundle install
```

Configure gitlab-pgrepack

The configuration file is located at `~/gitlab-pgrepack/config/gitlab-repack.yml`. Main parameters to check/modify are:

- **host**: must point to the cluster leader (warning: you must not connect via pgBouncer; we need to control **statement_timeout**, so always connect directly to Postgres)
- **ratio_threshold**: Estimation of “how much bloat” is needed to be considered a **gitlab-repack** target. Any number between 60 and 80 is reasonable.
- **real_size_threshold**: minimum size (in bytes) to be considered a **gitlab-repack** target. Using 10000000 (10 MB) seems reasonable.

For fresh installs, you can configure the tool with this command (modify values as appropriate):

```
cat > ~/gitlab-pgpack/config/gitlab-repack.yml <<EOF
general:
  env: local
database:
  adapter: postgresql
  host: patroni-04-db-gstg.c.gitlab-staging-1.internal
  user: gitlab-superuser
  password: xxx
  database: gitlabhq_production
estimate:
  ratio_threshold: 80 # bloat ratio threshold in % (set to 0 for testing)
  real_size_threshold: 10000000 # real size of object in bytes (set to 0 for testing)
  objects_per_repack: 1
repack:
  command: PGPASSWORD=xxx pg_repack -h patroni-04-db-gstg.c.gitlab-staging-1.internal -p 5432

# Optional: Grafana annotations
grafana:
  auth_key: false # put API key here to enable
  base_url: https://dashboards.gitlab.net
EOF
```

Get objects that need to be repacked

The below command outputs both tables and indexes that will require repacking:

```
~/gitlab-pgpack/bin/gitlab-pgpack estimate
```

Depending on the values of

- **ratio_threshold**, and
- **real_size_threshold**

in your **gitlab-repack** configuration file, the output may vary, but in general, it will output the list of commands you need to execute. You can save them in a safe place, while you analyze the right order/time to do so:

```
gerardoherzig@patroni-04-db-gstg.c.gitlab-staging-1.internal:~$ ~/gitlab-pgpack/bin/gitlab
```

```
I, [2020-06-23T21:07:10.079325 #7531] INFO -- : SET statement_timeout=0
```

```
I, [2020-06-23T21:07:10.079882 #7531] INFO -- : SET idle_in_transaction_session_timeout=0
```

Note: This is based on an estimation of database bloat.

Note: Executing full table repacking also removes index bloat.

TABLE bloat:

64.64 GiB of bloat (24.5 % ratio of total size)

Recommended repacking for table bloat:

```
gitlab-pgpack repack --type=tables --objects=public.project_mirror_data
gitlab-pgpack repack --type=tables --objects=public.remote_mirrors
gitlab-pgpack repack --type=tables --objects=public.ci_build_trace_section_names
gitlab-pgpack repack --type=tables --objects=public.import_export_uploads
gitlab-pgpack repack --type=tables --objects=public.geo_event_log
gitlab-pgpack repack --type=tables --objects=public.ci_triggers
gitlab-pgpack repack --type=tables --objects=public.pages_domains
gitlab-pgpack repack --type=tables --objects=public.geo_repository_updated_events
gitlab-pgpack repack --type=tables --objects=public.suggestions
gitlab-pgpack repack --type=tables --objects=public.geo_node_statuses
Expected cleanup of 311.34 MiB of bloat.
```

INDEX bloat:

44.90 GiB of bloat (31.2 % ratio of total size)

Recommended repacking for index bloat:

```
gitlab-pgpack repack --type=btrees --objects=public.index_user_highest_roles_on_user_id_and
gitlab-pgpack repack --type=btrees --objects=public.user_highest_roles_pkey
gitlab-pgpack repack --type=btrees --objects=public.index_geo_reset_checksum_events_on_pro
gitlab-pgpack repack --type=btrees --objects=public.index_geo_event_log_on_container_repos
gitlab-pgpack repack --type=btrees --objects=public.geo_event_log_pkey
gitlab-pgpack repack --type=btrees --objects=public.geo_reset_checksum_events_pkey
gitlab-pgpack repack --type=btrees --objects=public.index_geo_event_log_on_reset_checksum
gitlab-pgpack repack --type=btrees --objects=public.index_merge_request_diffs_on_merge_req
gitlab-pgpack repack --type=btrees --objects=public.index_namespaces_on_runners_token
gitlab-pgpack repack --type=btrees --objects=public.index_namespaces_on_runners_token_enc
gitlab-pgpack repack --type=btrees --objects=public.users_reset_password_token_key
gitlab-pgpack repack --type=btrees --objects=public.index_users_on_group_view
gitlab-pgpack repack --type=btrees --objects=public.index_users_on_accepted_term_id
```

Script

- Open a tmux session before running this script.

pg_repack can generate lots of WAL.

Also, giving pg_repack more memory can speed up the process considerably.

```
sudo gitlab-psql -c "ALTER SYSTEM SET max_wal_size TO '16GB';"
sudo gitlab-psql -c "ALTER SYSTEM SET maintenance_work_mem TO '20GB';"
sudo gitlab-psql -c "SELECT pg_reload_conf();"

```

REPACKING GOES HERE

Example of an index

```
~/gitlab-pgpack/bin/gitlab-pgpack repack --type=btrees --objects=public.index_chat_names

```

Example for a table

```
~/gitlab-pgpack/bin/gitlab-pgpack repack --type=tables --objects=public.geo_node_statuses

```

Restore the previous settings

```
sudo gitlab-psql -c "ALTER SYSTEM RESET maintenance_work_mem;"
sudo gitlab-psql -c "ALTER SYSTEM RESET max_wal_size;"
sudo gitlab-psql -c "SELECT pg_reload_conf();"

```

Monitoring and best practices during repacking

General advice

- The main danger during repacking is **disk IO**. Repacking is very IO intensive; it reads and writes a lot. So do it during a low-activity period of time – preferably, during a weekend.
- Closely watch monitoring (see details below) and be ready to slow down the process, or postpone it.
- Repacking of tables is beneficial but might be difficult (locking issues, a lot of IO). Consider skipping tables for which repacking fails due to locking issues and “downgrade” the operation to repacking only indexes of such tables.

Disk IO

This is a must-have to monitor – check disks activity (first of all, **sdb**, where PostgreSQL data directory is located) to watch read and write IOPS, throughput, and latency. Additionally, closely monitor the general system performance – first of all, database request latencies (SQL query duration).

We still need to collect more experience and write more practical pieces of advice are to be written in this area (How to throttle if needed? What are the “limits” when we should take actions if things go south?). General advice: understand the limits and be ready to react to the increased latencies (SSD disk latencies above 10ms are considered as too high leading to user-facing performance degradation). Possible ways to react:

- If needed, feel free to interrupt repacking. It is safe to interrupt the process with SIGTERM signal, the tool has cleanup logic.
- Consider switching to process only indexes (repacking of indexes is the most beneficial in terms of performance and, sometimes, disk space saved).
- Tools like `ionice` and `renice` *may* be helpful to mitigate the impact here, but in practice, they do not always improve the performance picture.
- Introduce pauses between operations, split to work to batches. Do not push everything ASAP – usually, it is better to postpone the operation rather than to put the performance down.

Lags

Increased WAL generation rates may lead to the situation when replicas are lagging behind significantly. Monitor the lags and be ready to slow down or even postpone the operation, not to allow replicas to have too high lags (many GiBs and many seconds).

Locking

Locking activity must be properly monitored. Refer to the locking activity runbook for hints.

Both things can happen: `pg_repack` being blocked or `pg_repack` being blocking.

The first can be OK in many cases (for example, `VACUUM` running in the “transaction wraparound prevention mode” blocks `pg_repack` and doesn’t yield). You can just wait for `VACUUM` to finish or *not* use `--no-kill-backend` in your `gitlab-repack` configuration (with care).

`Pg_repack` should never block regular database activity. In the case of normal queries being blocked by `pg_repack` activity, `gitlab-repack` should be canceled immediately!

Activity

Most of the activity of `pg_repack` is to build the temporary table that will, eventually, replace the original (meaning that 2x space is required – if we are processing the table of 100 GiB, we need additional 100 GiB temporarily):

```
-[ RECORD 1 ]-----+-----
datid        | 241498
datname      | pgrepack_test
pid          | 24505
usesysid     | 10
username     | postgres
application_name | pg_repack
client_addr  | 127.0.0.1
client_hostname |
```

```

client_port      | 54102
backend_start    | 2020-06-16 11:24:34.604791-03
xact_start       | 2020-06-16 11:25:35.270385-03
query_start      | 2020-06-16 11:25:35.27519-03
state_change     | 2020-06-16 11:25:35.275191-03
wait_event_type  |
wait_event       |
state            | active
backend_xid      | 2481299
backend_xmin     | 2481299
query            | INSERT INTO repack.table_241537 SELECT id,"time",data FROM ONLY public.test;
backend_type     | client backend

```

Cleanup activity

Under some circumstances (e.g., if your session crashes), `pg_repack` may not have been shut down cleanly. Our wrapper, `gitlab-pgrepack`, has some auto-cleanup logic for such cases, but still, you may need to take some actions to clean up.

1. Check for trigger (called `repack_trigger`) on the tables you were working on:

```
pgrepack_test=# \d test
```

		Table "public.test"		
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('test_id_seq'::regclass)
time	timestamp without time zone			
data	text			

Indexes:

```
"test_pkey" PRIMARY KEY, btree (id)
```

Triggers firing always:

```
repack_trigger AFTER INSERT OR DELETE OR UPDATE ON test FOR EACH ROW EXECUTE PROCEDURE repack_trigger()
```

If that is the case, you can drop that trigger manually with:

```
drop trigger repack_trigger ON test;
```

1. You might also need to clean up the temporary table that `pg_repack` creates manually. First check for existence:

```
pgrepack_test=# \dt repack.
```

```
List of relations
```

Schema	Name	Type	Owner
repack	log_241537	table	postgres

(1 row)

Then, it is safe to drop the `repack` schema completely and re-install the `pg_repack` extension (dropping the table implicitly):

```
drop schema repack cascade;  
drop extension pg_repack;  
create extension pg_repack;
```

1. In rare cases, an advisory lock may be left by an interrupted repacking session – see such locks using `select * from pg_locks where locktype = 'advisory'`; and use `pg_advisory_unlock(..)` function to remove the stale locks.